MARVIN — Technical Reference

The exhaustive companion to the white paper: every subsystem, its logic, the decision record behind it, and pointers into the code. Written for contributors and deep evaluators. Covers v0.1.107 (2026-09-07). Where this document and the repository disagree, the repository wins.

> **Since v0.1.55.** Subsystems added after this document's original scope, each with its own decision record: the in-process **pty terminal** (ADR-0078), **background subagents** and built-in read-only agents (ADR-0080), **implementer subagents on isolated git worktrees** (ADR-0081 — the one amendment to the single-assistant rule), **Claude plan usage** from `rate_limit_event` (ADR-0082), the **escalating graph-drift rail** (ADR-0083), **blast-radius and pre-ship impact** queries (ADR-0084/0085), **dependency bootstrap and update checks** (ADR-0086), **Obsidian vault integration** and the plans canvas (ADR-0089 → 0092), the **advisor verdict parser** and its caveat record (ADR-0094/0095, amended 2026-08-31), and **provider-aware model resolution** for OpenRouter (ADR-0096). Two cross-cutting lessons are recorded as ADR-0097 (verify against the surface that *runs*, not the one that *reports*) and ADR-0098 (a rail keyed on vendor tool names is only as durable as those names). Since 2026-09-01: an **LSP client** whose diagnostics come from the buffer (ADR-0099), `/refine` proposing practice lessons (ADR-0101), **two sessions in one worktree** with the collision surfaced (ADR-0102), a **derived implementer-branch lifecycle** (ADR-0103), the **ship-review gate** at `git commit` (ADR-0104, §2.7) and the **practice loop** (ADR-0105, §2.7 and §4). Since 2026-09-07: **vertical-slice milestones** — a Phase 5 rule, a `relations` filter on `graph_path` (§3.3), and the `plan.horizontal` / `plan.vertical` kinds whose layers are discovered from the project's graph (§2.7, §4). v0.1.106, same day: the plan spine (`PlanModel.swift`) excludes a Definition-of-Done / Scope-of-Done / Acceptance-criteria block from step parsing and renders it verbatim, strips a leading checkbox from step ids so a plan re-seeded from its file keeps its progress, seeds `[x]` as completed, and heals an already-absorbed spine on the next reconcile (ADR-0052 / ADR-0068 lineage; 7 new tests, 641 assertions).

Paths are relative to the repo root. `runtime/` abbreviates `sidecar/packages/runtime/src/`. ADRs for **this repo** live at `docs/decisions/`; when MARVIN writes ADRs into a **user's project** they go to `<workDir>/docs/adr/`.

Vocabulary (Claude Agent SDK terms used throughout): `canUseTool` — the SDK callback invoked before every tool execution; MARVIN's gate lives there. `agentID` — present on tool calls made by a spawned subagent. `TodoWrite` — the SDK tool the model calls to maintain its task checklist. `Task {subagent_type}` — how a subagent is spawned. `disallowedTools` — SDK-level tool denial for an agent. Graphify terms: edges carry EXTRACTED / INFERRED / AMBIGUOUS confidence tags; "god nodes" are the highest-degree nodes (the de-facto architectural anchors).

---

1. Core loop & modes

1.1 The single-assistant loop

MARVIN is one Claude session in a continuous user↔assistant loop (ADR-0001). Workflow "roles" are phases the one assistant moves through, not agents that hand off to each other. This is the founding constraint everything else respects; the 2026 multi-agent literature it is based on reports up to 70% degradation on sequential code work and 17× error amplification in flat agent topologies.

1.2 The 8-phase workflow

Intake → Discovery → Impact analysis → Architecture → Plan → Implement → Verify → Ship, encoded in `runtime/personality.ts` (`CORE_BEHAVIOR`) and explained in `docs/concepts/eight-phase-workflow.md`. Phases are labelled in chat (`**[Phase N · Name]**`); mutating tools are forbidden before Phase 6; trivial changes take an explicit fast-path. Phase 6 carries a bounded self-remediation contract and Phase 7 a surface-and-offer contract (§11.3).

1.3 Modes: Ask / Agent / Plan

A `mode` axis orthogonal to the permission strategy (ADR-0036):

Mode	Mechanism	Effect
ask	`readOnly` flag in `classifyToolCall` + SDK `disallowedTools` backstop	every mutating tool hard-denied; reads and graph queries allowed
agent (default)	normal gate	full loop under the auto/gated strategy
plan	read-only planning turn on the **advisor/planner model**; plan presented inline; approval runs a **separate Agent turn on the executor model**	strategy approved before execution; re-planning doesn't arise because execution is not a planning turn

1.4 The plan spine

The plan subsystem is MARVIN's most-revised area; the current design:

- **Plan card** — a plan reply must open `# Plan — <title>`; the native renderer shows it as a collapsible card; detection is content-shaped so it survives transcript replay (ADR-0036 addendum).
- **Two tiers** — a bare `TodoWrite` checklist renders as a neutral **Task list**; an approved plan renders as a purple **Plan —** `<title>` strip that persists and ticks off in place (`macos/MARVIN/TodoListView.swift`).
- **Reconcile, don't clobber** (ADR-0046) — the active plan owns hierarchical `PlanStep`s; an incoming `TodoWrite` reconciles into them (matched step → status update, unmatched item → nested sub-task) instead of replacing the list. Completion is computed over top-level steps only. Plans live in a revision-aware session list.
- **Join keys + roll-up** (ADR-0049) — the executor tags every `TodoWrite` item `[N]` / `[N.M]`, giving a stable join immune to rewording; a step with sub-tasks completes **iff** every sub-task completes (hard invariant, v0.1.50).

- **Persistence** — a presented plan is auto-written to `<workDir>/.marvin/plans/<slug>.md` and opened in the editor; `PlanFile.render` overlays `[x]` on completed steps and appends discovered work, re-persisted on every reconcile. On session load, `replay` reconstructs plan + checklist from the transcript.
- **Continue anchoring** (ADR-0050) — the Continue control injects the active plan's concrete steps plus a guardrail: resume *only this plan*, no re-auditing.
- **Plan-in-context** (ADR-0051) — the client sends an authoritative `planContext` snapshot every turn; the runtime appends it as a `<system-reminder>` suffix on the user message (the volatile, uncached tail — prompt-cache-safe, never persisted), so compaction cannot lose the plan.

Code pointers for the spine: the plan model, reconcile logic, `PlanFile` rendering, and `replay` reconstruction live on the app side in `macos/MARVIN/ChatPreviewView.swift`; the two-tier strip and plan card are `macos/MARVIN/TodoListView.swift` and `PlanCardView.swift`; the per-turn `planContext` injection is in `runtime/sdk-runner.ts`.

1.5 Interactive decisions

When the model reaches a genuine fork it calls `AskUserQuestion`; the gate routes it through the confirm channel (§2.1) and the native `AskQuestionSheet` renders clickable options (single/multi + "Other"), returning the pick as the tool result (ADR-0040). The sheet registers with **no auto-deny timeout** — a human weighing a decision is never silently overridden (v0.1.40 fix).

1.6 Session history

Turn transcripts hydrate tail-first: the last 200 events paint instantly; "show 200 earlier / show full log" pages the rest in with an "N of M" count (ADR-0048). The 120MB worst case is reachable but never auto-loaded.

---

2. Permission & safety

2.1 The structural gate

`canUseTool` (`runtime/sdk-runner.ts`) runs **before** every tool execution. Both permission strategies flow through the same classifier, `classifyToolCall` — there is no second policy hidden in a closure (ADR-0015):

- **auto** — everything not hard-denied runs; every mutating call is still classified and written to a JSONL audit log (`runtime/auto-audit.ts`, read via `GET /api/audit`). Bypass-only allowances are tagged `auto-mode bypass:` so a later audit can see what gated mode would have prompted on.
- **gated** — three-way classification: auto-allow (reads, whitelisted commands), confirm (Edit/Write/unlisted Bash → a confirm card with the exact diff), hard-deny.
- **Hard-deny floor** — destructive shell patterns (`rm -rf /`, force-push to main, credential-file writes) short-circuit in **both** strategies.

2.2 The subagent read-only invariant

Any tool call carrying an SDK `agentID` that would mutate the workspace is hard-denied in `classifyToolCall` (ADR-0030). This single choke-point is what makes every subagent pattern (§5) safe; it cannot be configured off. Precision on "would mutate": denial of the file-editing tools (Write/Edit/NotebookEdit) is structural — the tool identity is unambiguous — while mutating *Bash* is the gate classifier's judgment (pattern lists + policy), a distinction worth keeping in mind when evaluating the guarantee. Contrast §2.7's design hooks, which *are* configurable; this invariant and the hard-deny floor are not.

2.3 Change checkpoints & review

The gate snapshots each file's pre-image the first time a session touches it (`runtime/change-checkpoints.ts`, disk-backed under `<dataDir>/checkpoints/`) — ADR-0034. The review window diffs against that snapshot (not git HEAD, which would show the user's own uncommitted work as MARVIN's). Accept advances the baseline; **reject reverse-applies the diff**, restoring the user's uncommitted state — something `git checkout` could not do. Committing clears reviewed-clean files (`reconcileCommitted`). Known v1 blind spot, documented in the ADR: Bash-driven mutations are not pre-imaged.

The review surface is a dedicated resizable window: side-by-side with Split/Inline toggle, single-column rendering for added/deleted files, virtualized rows, and >1500-line diffs gated behind "Show anyway".

2.4 The file-system write channel

User-initiated tree operations go through a second gated channel (ADR-0008): `fsWritePolicy` (`sidecar/packages/tools/src/fs-write-policy.ts`) classifies seven ops. Delete-to-Trash is auto (reversible); permanent-delete and secret-file writes (`.env*`, `*.pem`, `id_rsa*`) are confirm-danger; writes cap at 5MB. Confirm tokens are session-scoped, one-shot, 60s TTL. Both read and write routes share `fs-sandbox.ts`: canonicalization, symlink + ancestor-symlink escape rejection, path-length and NUL checks. OS→tree uploads require the `X-Marvin-Client` header plus size caps (ADR-0009).

2.5 The git channel

All git mutations flow through `sidecar/packages/git/` (ADR-0012): `runGit` is the only place that shells to git (`shell:false`, `GIT_TERMINAL_PROMPT=0`, timeouts, output caps); `argv-guards` whitelist every ref/pathspec/remote/message and reject flag-injection vectors (`-c`, `--exec-path`, `--upload-pack`, …) as a last line against RCE. `gitWritePolicy` classifies each op: `push --force` to main is always hard-denied; `--force-with-lease` is confirm-danger; dirty-tree branch switches and pulls are denied. Credentials are **inherited, never handled** (ADR-0013): MARVIN never writes child stdin, never transforms remote URLs, never prompts for git credentials; the user's own helpers (osxkeychain, gh, 1Password) answer. Remote stderr is classified into a stable error taxonomy for the UI.

2.6 Background-execution denial

Shell backgrounding (`cmd &`, `nohup`, `setsid`, `disown`) and the SDK's `run_in_background` are gate-denied (ADR-0038, ADR-0032) — a detached process would finish unreported. The honest alternative is §9.1.

2.7 Runtime design hooks

`runtime/design-hooks.ts` enforces the most load-bearing behavioral rules **structurally**, below the prompt. Four are built in: a blind source-file read on a codebase question is denied until the graph is consulted (graphify-first); a stale graph escalates a nudge (graph-drift, ADR-0083); an edit in security/schema/CI/migration paths is denied until an advisor consult has happened (advisor-on-ADR); and, since v0.1.102, a `git commit` is refused until the review skills the personality requires have run (`checkShipReview`, ADR-0104). The ship-review gate reads the diff the commit seals — the index plus whatever the same command stages: a boundary path (auth / creds / CI / sudoers / `.env` / `*.sh` / migrations) requires both `pr-review` and `security-audit`, more than three files or fifty lines requires `pr-review`, docs-only and lockfile-only pass; a review this turn covers the turn, an earlier one holds until the next commit; two denies per skill per turn, then allow and log; fails open on git errors.

Practice rules (ADR-0105) are the fifth family and the first driven by data rather than code. `checkPracticeRules` reads the rules the user accepted in the Practice pane from `~/.marvin/practice/<projectId>/` (mtime-cached) and enforces each at its tier: `prompt` (a line in the system prompt, via `practicePromptBlock`), `nudge` (an advisory line on the tool result) or `deny` (a refusal, offered only for kinds with a machine-checkable discharge). Since v0.1.103 the four built-in gates are themselves rows in that store (`builtin:*`), so their tier, on/off state and message are editable from the pane; when no row exists the gate runs natively, which is why the 93 gate tests did not move. ADR-0104's brakes apply to every family: a deny needs a discharge path, `measure` mode logs instead of denying, two denies per rule per turn, and a bypass is logged — since v0.1.107 the graphify-first and advisor-on-ADR gates included (`BUILTIN_GATE_MAX_DENIES`; they had been one-shot), with the bypass counted on the gate's row. v0.1.107 also made `regressed` a rate (two recurring sessions at half the pre-acceptance rate), added an evidence threshold to the proposal test, moved the runner to `runPracticeAsync`, and shipped extractor v7 after an audit found five of six regressed rows on a real project to be the loop misreading its own gates (ADR-0105 addendum). Since v0.1.104 three more MARVIN-owned hooks came out of the first practice report: a bare project-local skill name is rewritten to its plugin-namespaced form at the gate (the ADR-0058 `updatedInput` mechanism), a `PostToolUseFailure` hook remembers a failed Bash command for the turn so an identical re-run gets an advisory nudge, and the turn-close hook blocks, once, a three-plus-edit turn under an open plan that never updated `TodoWrite`. Since v0.1.105 the loop measures **plan shape**: `plan.horizontal` (a plan of three or more `[N]` milestones each confined to one area of the project, none crossing two) against `plan.vertical` (some milestone crosses, or is marked as a slice), with a `nudge`-tier template that fires on the `TodoWrite` presenting a horizontal plan via a `planShape` trigger. The areas are **discovered** by `discoverAreas` (`graphify-bridge/src/plan-areas.ts`): the project's code files, made relative to their common root, split recursively wherever a directory holds two or more substantial groups (≥ 3 files and ≥ 5 %, depth ≤ 3, dot-directories and test folders folded into their parent); each area's vocabulary is its own directory names and file stems, kept where ≥ 60 % of a token's occurrences are in that area and that share beats the area's share of all files by 0.2. A milestone crosses layers only on *structural* evidence — directory-level words from two areas, or a methodology

marker (end-to-end, tracer bullet, thin slice) — and steps that open with verify / test / lint / ship / commit / docs / ADR count for nothing. No graph, or fewer than two areas, means no verdict. MARVIN ships no layer vocabulary. The same release fixed two extractors from the day's report at their source: a source read is now what the gate says it is (`bash-search.ts` holds the one classifier both use), and a review's report excludes the CLI-echoed skill body and stops at the commit (extractor v6).

Under the default `enforce` mode a deny is unconditional; `measure` logs what would have been denied; `off` disables the layer (`MARVIN_DESIGN_HOOKS`). These rules are belt-and-suspenders: prompt contract (§3.1, §5, §11) *plus* runtime hook — unlike the §2.2 invariant and the deny floor, which have no off switch. This layer exists because audits showed prompt-only rules decay as prompts grow: the 2026-09-02 session audit that produced the ship-review gate found the review skills invoked 0× across eight pushes of CI, sudoers and credential changes while every mechanical rule in the same session held.

---

3. Context & knowledge

3.1 Graphify-first

For structural questions the graph is queried **before** files are read — a hard rule with per-tool MUST triggers in `personality.ts`, no judgement call (`docs/concepts/graphify-integration.md`). Measured 27.5× cheaper per structural question (`graphify benchmark`, 2026-08-15; an earlier ~36× figure was an estimate, never measured) than file-reading; answers cite `file:line`.

3.2 Two graphs, three scopes

Per project (ADR-0028): a **code graph** (`graphify-out/graph.json`, AST-extracted) and a **knowledge graph** (`graphify-out/knowledge/graph.json`, heading structure + cross-links of docs/ADRs/memory). All six MCP tools accept `scope: "code" | "knowledge" | "all"` (default `"code"`).

3.3 The `marvin-graph` MCP

In-process server (`sidecar/packages/graphify-bridge/`), read-only, blanket-allowed at the gate: `graph_summary`, `graph_search`, `graph_neighbors`, `graph_query`, `graph_path`, `graph_save_result`. Edges carry EXTRACTED / INFERRED / AMBIGUOUS confidence tags; god nodes and communities are first-class. `graph_path` (undirected BFS, in-process) takes a `relations` filter — `["calls", "imports", "imports_from", "method", "contains"]` for a path code actually takes rather than one threaded through a `references` string mention — and every hop carries the relation of the edge it leaves by and its `sourceFile`; the rendered arrows were off by one hop before 2026-09-07. This is Phase 5's tracer-bullet tool: entry symbol → persistence symbol is the first milestone's touchpoint list.

3.4 Lifecycle & the context budget

While the IDE has a project open, `/api/chat` fires debounced, fire-and-forget AST rebuilds of both graphs each turn — scoped to the active project, never MARVIN's own repo, zero LLM cost (ADR-0041). The same ADR budgets first-message context: `buildProjectContext` (`sidecar/packages/project-context/`) injects a graph header, the **ADR titles index** (not bodies), the **memory tail** (not the log), curated docs whole, and open backlog items. Measured effect: ~566K → ~13.4K tokens on a mature production project.

3.5 Project fingerprint & infra probes

`fingerprint.ts` derives ~10–42 namespaced tags (`framework:next@16`, `architecture:multi-tenant`, `compliance:gdpr`, `test:playwright`) from manifests and code tells, cached at `.marvin/fingerprint.json` (ADR-0024); it drives skill enablement and recommendations (§6). Project-agnostic infra probes (`probeHttp`, `probeDockerContainer`) back Phase 2's "probe running infra" without any hardcoded service list. Both files live in `sidecar/packages/project-context/src/`.

---

4. Cross-session persistence

Content class determines the store; each store has an **enforced write path** that rejects the wrong class:

Class	Store	Write path	ADR
Durable facts (invariants, gotchas, constraints)	`.marvin/memory/<slug>.md` + one-line index	`remember` MCP tool (caps, supersede-by-name, rejects activity/status)	0042
Deferred actionable work	`.marvin/backlog/<slug>.md` + index	`backlog_add` / `backlog_resolve` (consent-gated; `provisional:true` auto-parks at discovery)	0044, 0047
Material decisions	`<workDir>/docs/adr/NNNN-*.md` (user projects; this repo uses `docs/decisions/`)	Phase 4, enforced template with Scope-of-Done	triggers in §11.4
Status / activity	git history, changelog	ordinary commits	—
Session notes	`.marvin/session-notes.md`	the native Scope-met chip	0042
Behaviour — repeat failures, the rules about them, whether they held	`~/.marvin/practice/<projectId>/` (findings ledger, rules, runs; config and score weights shared across projects)	the Practice pane only, over `/api/practice`: approve / dismiss / fixed / escalate / adopt. The nightly runner writes findings, never rules	0105

On the backlog's two-step capture: `provisional:true` auto-parks a *memo* the instant something is noticed (no go-ahead needed — a memo is not a commitment); the consent gate is the keep/dismiss review at the scope-met handoff. Capture is automatic, retention is consented.

The practice store is the one layer under the data dir rather than the project's `.marvin/`, because it is about MARVIN, not the project. A finding is keyed by a fingerprint (`kind[:qualifier]`) that a deterministic extractor computes over a transcript, counted by *distinct session*, with day-two semantics (new / recurring / proposed / regressed / confirmed / dismissed-then-resurfaced) and a versioned extractor (v6 at v0.1.105). Twenty-two kinds ship, eighteen of them in nine failure/success pairs so a finding carries a rate; three are report-only (cache re-creation, repeated errors, over-budget turns) because they are about MARVIN's implementation, not a behaviour a rule can change. Scoring is a small linear model over recurrence, cost, rate, reliability and actionability, minus decay; the weights can be fit from every ledger's own outcomes (Spearman rank correlation, coordinate descent; cost-share ranking below eight labels) and are applied only on a click, with provenance. A project with fewer than three sessions cannot propose; it shows a calibration counter and offers rules confirmed in the user's other projects for adoption, scoped to this project with its own verification clock. Subagent calls are never counted. No model reads a transcript; the one model call in the pane drafts a rule's wording from aggregates, in a read-only two-turn session.

The design was forced by a real failure: a memory file activity-logged to 419KB (~~99% redundant with ADRs/git) that overflowed the context window.~~ `~~/memory-compact~~` ~~migrates legacy logs. The backlog is a **parking lot, never a queue** — no agent pulls from it (Golden Rule 1); open items surface in the next session's context and in the macOS backlog panel. Transcripts (~~/.marvin/sessions//*.jsonl`~~ `~~, every event verbatim) and the cost ledger (~~` ~/.marvin/cost-tracker.json`, daily/weekly/lifetime per project) complete the picture.

---

5. Subagents

Three sanctioned patterns, all read-only by the §2.2 invariant, all bounded; anything else is forbidden by ADR-0001:

- **Advisor** (ADR-0007, ADR-0033) — a registered agent definition carrying the user's advisor model and its own reasoning effort; spawned via `Task {subagent_type:"advisor"}`. Blunt-critique structure: risks / alternatives / pushback / verdict. Eight MUST triggers (ADR writes, security-sensitive work, blast radius ≥5, non-backward-compatible changes, architecture tie-breaks, concurrency, crypto, user request) plus anti-triggers.
- **Scout** (ADR-0014) — breadth-first read-only research (`disallowedTools` at the SDK level + the gate invariant); MUST for 3+ parallel searches or context relief; MUST NOT be sequential-implementation or user-facing authority.
- **Dynamic workflows** (ADR-0030) — script-orchestrated parallel read-only fan-outs for audits / surveys / migration discovery; opt-in (high effort or explicit ask), never automatic, never implementation.

A standing pattern rather than a type: after drafting an ADR, an advisor "future-MARVIN critique pass" lists every question the ADR leaves unanswered; gaps are closed before the user sees it.

---

6. Skills

- **Installed ≠ active** (ADR-0037) — `skill-enablement.ts` computes the active set: explicit user choice, else core skills ∪ fingerprint-matched domain skills. The prompt names the active set each turn and tells the model to ignore the rest (measured 20 → 7 relevant on MARVIN's own repo). Per-skill toggles in the Skills pane persist to `.marvin/skills.json`.
- **Deterministic triggers** — each core skill (test-driven-development, systematic-debugging, pr-review, security-audit, frontend-design, graphify) has enumerated MUST / MUST-NOT trigger lists in `personality.ts` with no judgement escape hatch in the wording. To be precise about the guarantee: these are prompt-level contracts — enumerated form measurably outperforms soft language, but only the runtime layers (§2) are deterministic. The redesign followed a 2026-05-22 transcript audit that found five of six skills had fired approximately zero times across thousands of qualifying turns.
- **Skill audit** (ADR-0024, ADR-0025) — a `## Skill audit pending` block injects until `.marvin/skills.md` exists; MARVIN owes exactly one chip-strip recommendation with two verbs never mixed: **install** (user-global, `~/.claude/skills/`) for language/framework/test tags, **build** (project-local, `.marvin/skills/`, PR-reviewable, via skill-creator's eval loop) for architecture/domain/compliance tags.
- **Fetch from Git** (ADR-0039) — "Add from GitHub" shallow-clones a repo / sub-path / plugin marketplace, discovers `SKILL.md` folders, installs; clone-and-copy only, never executes repository code.

---

7. Models & auth

- **Role routing** (ADR-0002, ADR-0003) — independent executor and advisor/planner model slots with per-role reasoning effort (ADR-0033); a Sonnet executor + Opus advisor combination cuts inference cost substantially on routine work (~30–40% by per-token pricing arithmetic; workload-dependent, not a benchmark). Plan mode runs on the planner slot, execution on the executor (§1.3).
- **Auth resolution** (`runtime/auth.ts`, `auth-config.ts`) — the supported credential is an **API key**: the key configured in Settings (a `0600` file at `~/.marvin/auth-config.json`, provider `anthropic` or `openrouter`) wins; otherwise env `ANTHROPIC_API_KEY`. Raw keys are never logged and never returned by any status surface (last-4 hint only). OpenRouter is reached by pointing the CLI at a local proxy (`/api/proxy/openrouter`) that forwards Anthropic-format requests, and every model id is then OpenRouter's (ADR-0096).
- **Not a supported credential: a Claude.ai subscription login.** Anthropic's authentication policy (Legal and compliance, February 2026) reserves OAuth sign-in for Claude Code and Anthropic's own apps and requires API keys for products built on the Agent SDK. The older host-credentials detection and the Keychain read for model discovery (ADR-0029) predate that policy; they are legacy, undocumented here, and not supported.

---

8. UI surfaces (macOS SwiftUI app)

The app (`macos/MARVIN/`) is fully native — the Tauri/WebView era was migrated out ship-of-Theseus style (ADR-0010/0011/0016 record the history). Major surfaces:

- **IDE shell** — three panes (file tree / chat / brain-graph), split-view autosave, light-first OKLCH theme with dark mode (ADR-0006).
- **Chat** — mode + effort controls in the input (`ChatModeToolbar`), per-project persisted tabs, image paste + attachments, plan cards, the two-tier checklist strip, decision sheets, compaction banner (ADR-0022), scope-met chip strip.
- **Editor** — STTextView with tree-sitter highlighting (Swift, TS/TSX, JS/JSX, Go, Rust, JSON), diff gutter positioned from real layout fragments, ⌘S with mtime compare-and-swap.
- **Navigation** — Quick Open (⌘P), graph-backed Go-to-Symbol (⌘T), ripgrep Find-in-Files with replace-all, file history popover, Quick Look.
- **Terminal** — PTY-backed with ANSI parsing; build-task palette (⌘⇧B) discovering tasks from package.json / Makefile / Package.swift / Cargo.toml; diagnostics panel parsing compiler output.
- **Source control** — stage/commit/push/pull/fetch through the guarded git channel (§2.5), branch line, remote-error banner with the stderr taxonomy.
- **Review window** — §2.3.
- **Backlog panel** — browse / Done / Dismiss / Promote-to-plan (which switches to Plan mode and queues if a turn is live — v0.1.53).
- **Context panel** — the status-bar `ctx` chip opens a live breakdown: exact resident/window % from SDK usage plus per-category estimates (system prompt · tools/MCP · project context · transcript · free).
- **The brain** (ADR-0019) — a Metal-rendered live state indicator with five behavioral states (idle / thinking / tool / writing / error).
- **Status bar** — health hairline, cost pill, branch badge, model row, graph-ops ratio.

8.1 Health & resilience

`HealthMonitor` demotes to `.offline` only after **3 consecutive** `/api/health` misses (5s timeout, fast re-poll while misses pend) — one slow poll from a busy sidecar no longer tears down and rebuilds the IDE (v0.1.54). One live turn per session is enforced server-side (`POST /api/chat` → 409 rather than evicting; Stop is authoritative via `cancelLiveTurn`); closing the window doesn't kill a running turn (resume via `GET /api/chat/resume`).

---

9. Background & async

- **Background jobs** (ADR-0038) — `run_background_job({command, reason})` — the tracked replacement for the raw backgrounding mechanisms §2.6 denies (similar name, opposite fate: `run_in_background` is denied, `run_background_job` is the sanctioned tool) — spawns a tracked child, streams an 8KB output tail, and **fires a real follow-up turn on process exit** with the exit code and tail. Limits: ≤3 concurrent per session, chain depth ≤8. Shutdown signals (SIGTERM on app quit) are "stopped, not finished" — no spurious failure turns on relaunch (v0.1.48).

- **Wakeups** (ADR-0031) — `schedule_wakeup` re-invokes MARVIN at a chosen time via a persistent, boot-re-armed scheduler (delay 60s–24h, ≤5 pending, re-schedule depth ≤3); a fired wakeup **yields** to a live interactive turn and re-arms.
- **Announce SSE** (ADR-0043) — an always-on per-project stream re-attaches an idle client to any server-initiated turn (job completions, wakeups), with a "background job running" chip.
- **git-watch** (`sidecar/packages/git-watch/`) — per-workDir commit detection feeding graph auto-rebuild and inline commit surfacing.
- **The anti-fabrication contract** (§11.5) makes these three mechanisms the *only* honest ways to promise future work.

---

10. Distribution & operations

- **Homebrew** (ADR-0023) — `brew tap RobertIlisei/marvin && brew install --cask marvin-ai` (token disambiguated from the unrelated "marvin" cask). The .app bundles the Next.js standalone sidecar + a pinned Node 22 runtime; the SwiftUI process spawns and reaps it, reclaiming port 3030 first (ADR-0035) so "new app on disk, old code in memory" cannot recur.
- **Signing** (ADR-0026) — every release zip carries a minisign (EdDSA) signature; the public key is pinned in the app repo, the tap README, and the cask constant — three copies across two repos, so tampering with one is visibly inconsistent. Cask auto-verify is the ADR's Phase 2.
- **macOS 26 Gatekeeper** (ADR-0027) — ad-hoc-signed bundles are kernel-killed in `/Applications`; MARVIN installs to `~/Applications`, the cask strips quarantine in a `postflight`, and first launch needs the one-time "Open Anyway".
- **Lifecycle** — `bin/marvin start/stop/restart/status/logs/doctor/ knowledge-graph`; `install-macos-app --bundled` builds, bundles, health-probes on a probe port, then installs; `brew uninstall --zap` wipes `~/.marvin`.
- **Observability** — `/api/health` reports auth mode + version; the sidecar logs to `~/Library/Logs/MARVIN/sidecar.log`. Optional **Honeycomb OTEL export** (`runtime/honeycomb-telemetry.ts`): per-turn env injection so the SDK-spawned CLI emits spans to the *user's own* Honeycomb account — off unless the user configures it; keys redacted from all logs.
- **CI/release** — tag push → `release.yml` builds the .app on a macOS runner, stamps the version into Info.plist, zips, signs, publishes; the cask is then bumped with the published sha256.

---

11. Behavioral contracts (`personality.ts`)

The prompt is a contract surface, not vibes. The catalogue (each item a MUST list + MUST-NOT list + narrow fallback test):

1. **Phase gating** — labelled phases, stops between them, no mutation before Phase 6, explicit trivial fast-path.
2. **Definition of Done / match-not-improve** — 3–5 falsifiable bullets before code; Phase 7 verifies those exact bullets; adjacent improvements are surfaced ("noticed in flight, not in scope") and parked or asked — never silently landed. Real-work turns end `**Scope met:** … Anything else, or should I stop?` plus a machine sentinel for the UI chip strip.

3. **Verify-then-remediate** (v0.1.55) — mechanical failures (typecheck/tests/build) self-remediate ≤3 attempts per milestone with an early stop on identical consecutive errors; MUST NOT claim landed, weaken the DoD, or skip the failing check. Unmet DoD bullets get surface-and-offer: the gap + the one next step, then a gate. A blind retry-until-done loop was deliberately rejected.
4. **Deterministic ADR triggers** — nine MUST-write categories (foundational framework, public API shape, persistent-state schema, security boundary, default-model change, new MCP server, cross-cutting constraint, superseding, user-named) + anti-triggers + the "would future-MARVIN re-derive this in 8 weeks?" test.
5. **Async anti-fabrication** — never narrate a watcher that wasn't armed; the only honest options are block-and-wait, a background job, a wakeup, or handing back.
6. **Post-PR loop** — own the green build: run tests locally, one structured PR comment per run, fix the code under test, cap three run-fix-run cycles, name flakes honestly.
7. **Workflow health** — an injected audit block (Mode A propose / B execute / C defer) catches an in-flight project up on missed phases; a greenfield playbook adapts Phases 2–3 for empty repos.
8. **Personality** — `marvin` (dry wit, always delivers) vs `neutral`, a style layer never a refusal layer; ground-truth facts are placed first in the prompt (the highest-attention slot).

---

12. API & MCP surface

12.1 HTTP API (sidecar, `localhost:3030`)

Route groups under `sidecar/src/app/api/`: `chat` (+ `announce`, `resume`, `cancel`), `confirm`, `changes`, `files` (tree / content / raw / status / write / upload), `git` (status / diff / branch / log / push / pull / fetch), `graph`, `projects`, `sessions`, `skills` (+ `add`), `models`, `auth`, `health`, `context`, `cost`, `audit`, `backlog`, `terminal`, `honeycomb`, `worktrees`, `practice` (+ `run`, `findings`, `rules`). Details: `docs/reference/api.md`.

12.2 MCP servers

Server	Type	Tools	Gate posture
`marvin-graph`	in-process	6 graph tools	allow (read-only)
`marvin-memory`	in-process	`remember`, `recall`	allow (content-class enforced)
`marvin-backlog`	in-process	`backlog_add/list/resolve`	allow (content-class enforced)
`marvin-control`	in-process	wakeups + background jobs	allow (bounded by rails)
`playwright`	external stdio, **opt-in**	`browser_*`	observation auto · interaction confirm · `browser_run_code_unsafe` **deny**; scouts get observation only (ADR-0045)

Browser automation defaults to the Playwright **CLI** (one-shot screenshots, scripted checks, full `playwright test`); the MCP is for stateful navigate→snapshot→assert flows, off by default because a browser subprocess per turn is heavy. MCP-vs-CLI selection is itself a deterministic trigger in

`personality.ts`.

---

13. Deliberate non-goals

Explicit product boundaries, each with a recorded reason (`docs/roadmap.md` "Not planned"):

- **No multi-agent implementation** — the founding bet (ADR-0001); the three read-only subagent carve-outs are not a precedent.
- **No cross-project memory** — projects are isolated workspaces (ADR-0005); a new project starts from zero, by design.
- **No hosted service / shared state** — local-first is the trust model, not a phase.
- **No cross-platform desktop (yet)** — fully-native macOS is the current commitment; x86_64 and other platforms wait for demand.
- **No auto-model heuristics** — the user routes roles to models explicitly; MARVIN doesn't guess task complexity (ADR-0002).

---

14. Building & contributing

This document is the architecture map; the practical loop lives in the repo:

- **Dev setup & build** — `README.md` ▸ *Development setup* (sidecar: pnpm + Next.js dev on `:3030`; app: `swift build` or xcodegen + Xcode in `macos/`) and ▸ *Dev loop* for running both together. `bin/marvin install-macos-app --bundled` produces the installable .app.
- **Tests** — TypeScript: `npx vitest run` from the repo root (policy, sandbox, registry, and dispatch contracts are the best-covered areas); Swift: `swift test` in `macos/`. Honest scope note: the SDK loop, the UI shells, and most API routes are opportunistically covered at best — `docs/development/testing.md` has the current map.
- **Conventions** — work lands on `development` and fast-forwards to `main`; a gitleaks pre-commit hook is installed from `scripts/hooks/pre-commit`; CI is `.github/workflows/release.yml` (tag-triggered).
- **Understanding why** — read `docs/decisions/` (the 51 ADRs) before proposing structural changes, `docs/roadmap.md` for what's in flight, and `CLAUDE.md` for the golden rules any agentic session in this repo is bound by. Material changes should arrive with an ADR of their own.

Compiled against the 2026-07-02 full feature inventory (51 ADRs, v0.1.55). Corrections welcome — the ADRs are the authoritative record, and this document cites them everywhere it can.